

PortOne SDE II Assignment — Amazon Payments vs Settlement Reconciliation

Assignment Overview

As part of the hiring process for the Software Development role in PortOne's **Reconciliation product team**, we want to evaluate your practical skills in data ingestion, config-driven transformation, reconciliation and accounting-grade report generation.

This assignment mirrors a real product surface we run in production: reconciling Amazon's **Payments** report against Amazon's **Settlement** report for the same settlement period, and proving that the resulting numbers tie out to Amazon's own **Statement Summary**.

Amazon publishes the same settlement period through two different reports with different shapes and different granularity. Both are supposed to describe the same money. Your job is to ingest both, match them, roll them up into an accounting summary, and show that the summary agrees with what Amazon itself says.

What you will be given

#	Artifact	File	Notes
1	Amazon Payments report	amazon_payments_data.csv	One row per transaction line, amounts spread across typed columns (product_sales, selling_fees, fba_fees, total, ...)
2	Amazon Settlement report	amazon_settlements_data.txt	Tab-separated flat file, one row per amount component (amount_type /

#	Artifact	File	Notes
			amount_description / amount)
3	Payment mapping config	amazon_payment_configs_au_old.csv	See "Apply the mapping configs" below
4	Settlement mapping config	amazon_settlement_configs_au.csv	See "Apply the mapping configs" below
5	Sample output report	amazon_sample_output_report.xlsx	An example of the report shape we expect (structure reference, not an answer key)

All the files are shared here —

<https://drive.google.com/drive/folders/10z6zVOFAztK9PBNjADYekdhMxkQb1The?usp=sharing>

Objective

1. Ingest both files

Ingest the Payments CSV and the Settlement TXT into PostgreSQL.

- Ingest the files **as provided** — do not hand-convert, hand-clean, or pre-edit them outside your code. Whatever normalisation is needed must happen in your ingestion pipeline.
- Design your own schema. Document what you went with and why in your README. Keep the mapping configs themselves in tables too, so that the fixes in step 5 are data changes rather than code changes.
- Every ingested row must retain enough of its source data (**raw payload, source file, line number, or similar**) that any number in your final report can be traced back to the exact input rows that produced it.
- Important :: **Ingest both the files in a single table**, not in two different tables.

2. Apply the mapping configs

Both configs are the real production mapping rules for this marketplace/locale. They are what turn a raw file row into a reconcilable, summarisable record. Two columns matter most:

record_ref — a +-joined template for building the record's reconciliation key from the row. It is the **shared key**: it is constructed so that a payments row and a settlement row describing the same underlying money land on the same value.

to_summary_field_when_positive_amount / to_summary_field_when_negative_amount — which Summary sheet bucket the amount contributes to, chosen by the sign of the amount.

An empty value means the amount does **not** contribute to the summary (it is still ingested; it just isn't summarised through that rule).

Other notes on the configs:

- description / amount_description of any is a wildcard — it matches any value for that column.
- Config rows may have an empty transaction_type; treat those as the fallback/catch-all bucket.
- Matching is case- and separator-sensitive in ways the raw files are not always consistent. Normalising source values before matching them against config keys is part of the work.

Brownie Points:: If you can calculate the summary while the data is being ingested and not after the data is ingested.

3. Reconcile

Match records across the two sources on the shared record_ref.

- A record_ref present in both sources → **reconciled**.
- Present only in payments → **unreconciled payment**.
- Present only in settlements → **unreconciled settlement**.
- Note that the two files are at different granularity: a single payments row can correspond to several settlement rows for the same record_ref, and vice versa. Decide and document how you aggregate before matching.

4. Generate the Excel report

A single .xlsx with (at least) these two sheets:

Sheet Summary

Look at the Summary sheet in **amazon_sample_output_report.xlsx**. You have to figure out which line items belong on this sheet, how they are grouped and subtotalled, and how each one relates back to the summary fields the mapping configs route amounts into. Follow that structure.

Note: All the values in the Summary Sheet are deliberately set to zero, as the purpose of the sheet is to give you the structure of output required. Once you calculate the summary for all the records, you need to map and generate the Summary sheet correctly.

The two value columns must be derived independently: the Settlement column purely from ingested settlement records, the Payment column purely from ingested payment records.

Sheet Consolidated data

All records, in this order:

1. Reconciled records (payment side and settlement side of the match on the same row, with the difference)
2. Unreconciled payments, if any
3. Unreconciled settlements, if any

Include enough columns (record_ref, transaction type, description/amount type, SKU, date, settlement id, amounts, matched summary field, source) that a finance user can audit any line.

5. Find and fix the mismatches

This is the core of the assignment.

Check your generated Summary sheet. There are a small number of genuine mapping defects in the configs you have been given.

You are expected to:

1. Identify **which** summary lines differ and by **exactly how much**.
2. Trace each variance back to the specific record(s) and the specific config rule(s) that caused it — not guess.
3. Propose and apply a **mapping fix** (config-level, not a hardcoded correction in report code).
4. Re-ingest and regenerate the report.
5. Repeat until there are no mismatches in the Summary sheet.

Rules for this step:

- Fixes must live in the **mapping config data**, not in special-cased Go code. If you believe a fix genuinely cannot be expressed in the config schema, say so and explain what schema change would be needed.
 - No plugging numbers. A hardcoded adjustment that makes a total match is a failed submission.
 - Document every change: which config row, what it was, what you changed it to, why.
-

Technical requirements

- **Language:** Golang preferred. Any language is acceptable, but Go is what the team writes.
 - **Database:** PostgreSQL.
 - **Output:** Excel (.xlsx).
-

Submission checklist

-  **Source code** (GitHub link), with a README covering:
 - Steps to run the full flow end to end (ingest → reconcile → report)
 - The schema you designed and why
 - Assumptions made
 -  **A MAPPING_FIXES.sql** — the actual UPDATE/INSERT/DELETE statements that fix the mapping configs, one commented block per defect. Each block should state what the rule was, what you changed it to, why.
 -  **Generated Excel report** — both the *before-fix* and *after-fix* versions, so we can see the delta you closed.
 -  **PostgreSQL dump** (or a representative data sample) after ingestion.
 -  **A short progress log** — we care about how you got there, not just the final artifact. A running note (commits, a PROGRESS.md) showing what you tried, what you ruled out, and how you isolated each variance is explicitly part of the evaluation.
-

Evaluation criteria

Weight	Criterion
High	Accuracy of the final Accounting summary
High	Quality of the mismatch investigation — did you trace variances to root cause, or pattern-match until the number moved?
High	Correctness of config-driven ingestion (record_ref construction, wildcard handling, sign-based summary routing)
Medium	Reconciliation logic, including the granularity mismatch between the two files
Medium	Clarity and structure of the code; separation of ingest / recon / report
Medium	Auditability — can any number in the report be traced back to source rows?
Bonus	Error handling, idempotent re-ingestion, tests, performance on larger files

We are hiring for someone who will own this product area. The step we weight most heavily is step 5: finance reconciliation work is 20% pipeline and 80% "these two numbers differ by \$43.17 and I need to know exactly why."
